

VS CODE AND DOCKER

A full editor, wired into the container that builds your code

WHY THIS LECTURE

The gap left by lecture 2.7

You can already run a container and build inside it - but so far that means an interactive shell and typing `cmake / ./build/rooster` by hand. No syntax highlighting, no autocomplete, no clicking a line to set a breakpoint. This lecture connects Visual Studio Code, running on your machine, to that same container - so you get a real editor experience while every build still happens inside Linux, on GCC 16 or Clang 21.

What you'll install

- **Visual Studio Code** - the editor itself (separate from Visual Studio, despite the name).
- **Dev Containers** extension - `ms-vscode.remote-containers`. Lets VS Code talk to a container instead of your local filesystem.
- **C/C++** extension - `ms-vscode.cpptools`. IntelliSense and debugging.
- **CMake Tools** extension - `ms-vscode.cmake-tools`. Configure/Build/Debug buttons for a `CMakeLists.txt` project.

All three install the same way: **Extensions** view (Ctrl+Shift+X), search the name, **Install**.

WHERE DOCKER DESKTOP FITS

Still the same Docker as lecture 2.7

Dev Containers doesn't replace Docker Desktop - it drives it. Docker Desktop must already be installed and running (Windows: with the WSL2 backend enabled under **Settings -> Resources -> WSL Integration** if you're on Windows 10/11 Home). If `docker run` already worked for you in 2.7, you're ready for this lecture.

THE PLAN FOR THIS LECTURE

Two ways to connect, one we'll focus on

- Attach to a container you started yourself - the natural next step after 2.7, since you already know `docker run`. This is our main path today.
- `devcontainer.json` - a config file that lets VS Code create and manage the container *for* you. We'll look at it at the end as an optional upgrade once the manual flow makes sense.

STEP 1

Start a container, same as lecture 2.7

Nothing new here - this is the exact command from 2.7, run from the repo root:

```
# Windows (PowerShell)
docker run --rm -it -v "${PWD}\02.EnvSetup\2.8VSCodeAndDockerContainers:/workspace" masterclass-gcc:16
```

```
# Linux / macOS
docker run --rm -it -v "$(pwd)/02.EnvSetup/2.8VSCodeAndDockerContainers:/workspace" masterclass-gcc:16
```

Leave this terminal open - the container is now running, with this lecture's folder mounted at `/workspace` inside it.

STEP 2

Find it from VS Code

Open VS Code, then either:

- Command Palette (Ctrl+Shift+P) -> **"Dev Containers: Attach to Running Container..."**, then pick the container from the list, **or**
- click the **Remote Explorer** icon in the Activity Bar, expand **Containers**, and use the inline **Attach to Container** action next to the running container.

VS Code opens a **new window** connected to that container.

STEP 3

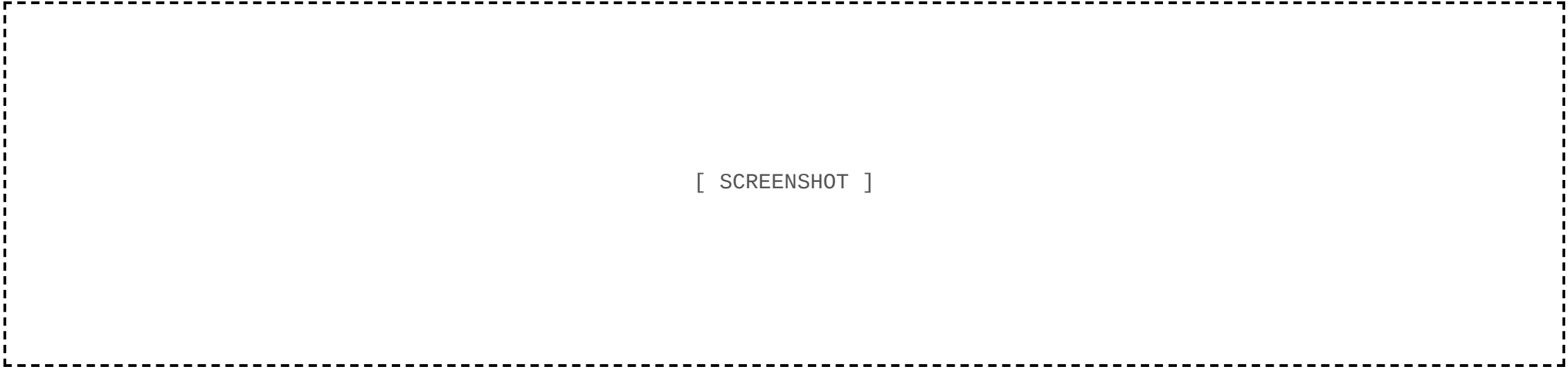
Point it at your code

That new window starts at the container's filesystem root, not your code. Use **File -> Open Folder...** inside that window and open **/workspace** - the exact folder you mounted with **-v** in step 1.

First time attaching to a container, VS Code generates a config file for it - **"Dev Containers: Open Container Configuration File"** lets you find it again later, e.g. to set **workspaceFolder** so this step happens automatically next time.

STEP 4

Install extensions - inside the container



[SCREENSHOT]

Extensions run on your machine and inside containers separately.

Install C/C++ and CMake Tools again from the Extensions view - notice the button now says something like "Install in Container".

Your local VS Code extensions don't automatically carry over.

STEP 5

Let CMake Tools find the compiler

Command Palette -> **"CMake: Select a Kit"**. CMake Tools scans the container's **PATH** for compilers - GCC 16 shows up automatically, no kit file to write by hand.

[SCREENSHOT]

STEP 6

Configure and build

Two commands, but now buttons instead of typing:

- **CMake: Configure** (Command Palette, or the status bar) - same as `cmake -B build -G Ninja .`
- **CMake: Build** - same as `cmake --build build`

Since this lecture's `CMakeLists.txt` only defines one target (`rooster`), there's nothing else to pick.

STEP 7

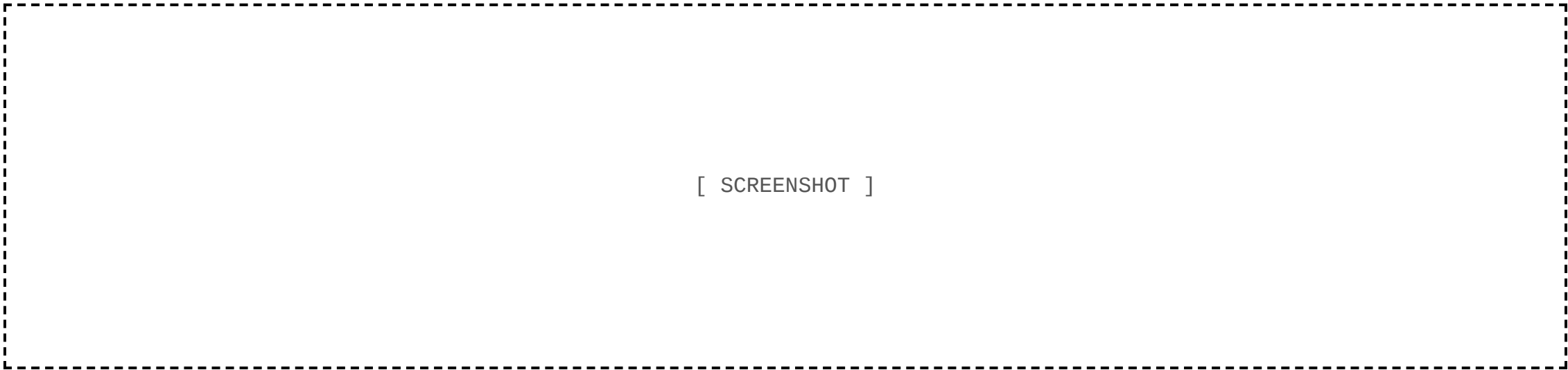
Run it

Click the **Run** button in the CMake Tools status bar, or Command Palette -> "**CMake: Run Without Debugging**". Output appears in VS Code's integrated terminal - still the container's terminal, just inside the editor now instead of the separate window from step 1.

STEP 8

Set a breakpoint and debug

Click in the gutter next to a line in `main.cpp` to set a breakpoint, then **CMake: Debug** (or **Shift+F5**). No `launch.json` needed for this basic case - CMake Tools uses `gdb`, already installed in this image, to drive the debug session.



[SCREENSHOT]

IF DEBUGGING REFUSES TO START

A specific error to recognize

Some Docker setups block the `ptrace` syscall gdb needs to attach to a process, surfacing as `ptrace: operation not permitted`. If you see it, re-run step 1's `docker run` with two extra flags:

```
docker run --rm -it --cap-add=SYS_PTRACE --security-opt seccomp=unconfined \  
-v "$(pwd)/02.EnvSetup/2.8VSCodeAndDockerContainers:/workspace" masterclass-gcc:16
```

These flags loosen container isolation - fine for a local course sandbox, not something to reach for on a production container.

CLEANING UP

Ending the session

Closing the VS Code window just disconnects the editor - the container (and the terminal from step 1) is still running. Since you started it with `--rm`, typing `exit` in that original terminal stops and removes the container in one step. Nothing to clean up by hand.

BASIC DOCKER COMMANDS WORTH KNOWING

You don't strictly need these today, but you will eventually

COMMAND	WHAT IT DOES
<code>docker images</code>	List images you've built or pulled (e.g. confirm <code>masterclass-gcc:16</code> exists)
<code>docker ps</code> (add <code>-a</code> for stopped ones too)	List containers
<code>docker exec -it <container> bash</code>	Open another shell into an already-running container
<code>docker logs <container></code>	See a container's output history
<code>docker stop <container></code>	Stop a running container
<code>docker rm <container></code> / <code>docker rmi <image></code>	Remove a container / an image

IMAGE VS. CONTAINER, ONE MORE TIME

The distinction that makes all of this make sense

An image (`masterclass-gcc:16`) is a fixed template - it never changes just by using it. A container is one running (or stopped) instance made *from* an image. You can start as many containers from the same image as you like; each gets its own throwaway filesystem layer on top, which is exactly why `--rm` is safe to use by default.

devcontainer.json - letting VS Code manage the container

Instead of running `docker run` yourself, a `.devcontainer/devcontainer.json` file next to a lecture's `CMakeLists.txt` lets VS Code create, mount, and tear down the container for you:

```
{
  "name": "C++ Masterclass (GCC 16)",
  "image": "masterclass-gcc:16",
  "workspaceFolder": "/workspace",
  "customizations": {
    "vscode": {
      "extensions": ["ms-vscode.cpptools", "ms-vscode.cmake-tools"]
    }
  }
}
```

Which one should you actually use?

Attach to Running Container

You already ran `docker run`. VS Code just connects - it never starts, stops, or forgets about the container.

Reopen in Container

VS Code owns the whole lifecycle - creation, mounting, extension install, and teardown - driven by one checked-in config file.

Both talk to the exact same images. Start with Attach since it reuses commands you already know - reach for `devcontainer.json` once you want one-click, repeatable setup for a folder you come back to often.